

Atlas — Manual Setup & Secrets Guide

Everything you must supply by hand — databases, keys, credentials and URLs — to run the Rust ledger platform locally and in production.

Event-sourced double-entry ledger · 5 services (gateway · identity · ledger · notification · worker) · Rust 1.96 workspace · Generated 2026-07-06

How configuration works. Every setting has three layers, later wins: (1) built-in defaults → (2) `config/{RUN_ENV}.toml` → (3) environment variables prefixed `APP__` with `__` as the nesting separator (e.g. `APP__DATABASE__URL`). **Secrets are supplied only via layer 3** (env / k8s Secret / secret manager) — never committed. Set `RUN_ENV=local` for dev, `RUN_ENV=production` in prod (which additionally rejects the insecure local defaults at boot).

1 · What you must provide (at a glance)

#	Item	Env variable	Used by	Required?	Example / source
1	PostgreSQL — identity DB URL	<code>APP__DATABASE__URL</code>	identity	REQUIRED	postgres://app:PASS@host:5432/identity_db
2	PostgreSQL — ledger DB URL	<code>APP__DATABASE__URL</code>	ledger	REQUIRED	postgres://app:PASS@host:5432/ledger_db
3	PostgreSQL — worker DB URL	<code>APP__DATABASE__URL</code>	worker	REQUIRED	postgres://app:PASS@host:5432/worker_db
4	Redis URL	<code>APP__REDIS__URL</code>	gateway, identity, ledger, notification, worker	REQUIRED	redis://:PASS@host:6379
5	Kafka / Redpanda brokers	<code>APP__KAFKA__BROKERS</code>	identity, ledger, notification, worker	REQUIRED	redpanda-0:9092,redpanda-1:9092
6	JWT RSA <i>private</i> key (PEM)	<code>APP__JWT__PRIVATE_KEY_PEM</code>	identity	REQUIRED (prod)	generate with OpenSSL — \$4
7	JWT public key delivery	<code>APP__JWT__JWKS_URL</code> or <code>APP__JWT__PUBLIC_KEY_PEM</code>	gateway	REQUIRED	JWKS URL (default) — \$4
8	Worker admin token	<code>WORKER_ADMIN_TOKEN</code>	worker	REQUIRED (prod)	random 32+ char secret — \$6
9	OAuth2 client id / secret	<code>APP__OAUTH__CLIENT_ID</code> / <code>__CLIENT_SECRET</code>	identity	OPTIONAL	from your OAuth provider — \$7
10	OpenTelemetry OTLP endpoint	<code>APP__OTEL__ENDPOINT</code>	all	OPTIONAL	http://otel-collector:4317
11	Public hostname + TLS cert	Ingress / <code>APP__JWT__ISSUER</code>	gateway, identity	REQUIRED (prod)	your domain + cert — \$8
12	Container registry	CI secrets / image refs	build & deploy	OPTIONAL	ghcr.io / ECR — \$9

Port map — gateway http 8080 / metrics 9100 · identity http 8081 / grpc 50051 / metrics 9101 · ledger http 8082 / grpc 50052 / metrics 9102 · notification ws 8083 / metrics 9103 · worker http 8084 / metrics 9104.

2 · PostgreSQL — three databases

One Postgres server hosts three logical databases, one per bounded context (ADR-0008). The app connects as a least-privilege `app` role, never the superuser. **notification is stateless** (Redis only, no DB).

2.1 Local (docker-compose)

Nothing to do — `deploy/postgres/init-databases.sql` creates the `app` role and `identity_db` / `ledger_db` / `worker_db` automatically on first `just up`. Default creds are `app:app` (local only).

2.2 Production (managed Postgres)

```
-- Run once against your managed instance as an admin user.
-- Use a STRONG generated password — do not reuse the local 'app:app'.
CREATE ROLE app WITH LOGIN PASSWORD '<STRONG_PASSWORD>'
    NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT;

CREATE DATABASE identity_db OWNER app;
CREATE DATABASE ledger_db   OWNER app;
CREATE DATABASE worker_db   OWNER app;
-- (repeat per db) revoke PUBLIC, grant CONNECT + schema ownership to app:
REVOKE ALL ON DATABASE identity_db FROM PUBLIC;
GRANT CONNECT ON DATABASE identity_db TO app;
```

Then set each service's `APP__DATABASE__URL` to its own database:

`identity` → `postgres://app:PASS@host:5432/identity_db`

`ledger` → `postgres://app:PASS@host:5432/ledger_db`

`worker` → `postgres://app:PASS@host:5432/worker_db`

schema is applied by the migrate job — see §10

Managed TLS: most cloud Postgres requires SSL. Append `?sslmode=require` to the URL if your provider mandates it.

3 · Redis & 3 · Kafka / Redpanda

Redis

Used for rate-limit token buckets, distributed locks, cache, Kafka-offset tracking, and notification pub/sub. Provide one URL:

```
APP__REDIS__URL=redis://:PASS@host:6379
# TLS: rediss://... · password via
APP__REDIS__PASSWORD=<secret>
```

Kafka / Redpanda

The event backbone. Provide the broker list:

```
APP__KAFKA__BROKERS=host-0:9092,host-1:9092
APP__KAFKA__CONSUMER_GROUP=ledger
APP__KAFKA__MAX_DELIVERY_ATTEMPTS=5
```

Topics are auto-created by the services; on locked-down clusters pre-create the per-context + DLQ topics.

4 · JWT signing keys (the auth chain)

Access tokens are RS256 JWTs. **identity** signs them with an RSA *private* key; **gateway** verifies them with the matching *public* key. This must be a **stable, shared** key in production — otherwise tokens die on restart and replicas reject each other.

4.1 Generate the keypair (once)

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out jwt_private.pem
openssl rsa -in jwt_private.pem -pubout -out jwt_public.pem
```

4.2 Give identity the private key

```
# PEM contents (PKCS#8 'BEGIN PRIVATE KEY' or PKCS#1 'BEGIN RSA PRIVATE KEY').
APP__JWT__PRIVATE_KEY_PEM="-----BEGIN PRIVATE KEY-----
... base64 ...
-----END PRIVATE KEY-----"
```

Enforced: outside `RUN_ENV=local`, identity *refuses to boot* without this — it will not silently mint an ephemeral per-pod key.

4.3 Give gateway the public key — pick ONE

Option	Set	When
A — JWKS fetch (recommended)	<code>APP__JWT__JWKS_URL=http://identity:8081/.well-known/jwks.json</code>	In-cluster. Gateway fetches & auto-refreshes the key (every <code>APP__JWT__JWKS_REFRESH_SECONDS</code> , default 300s). Nothing else to wire.
B — static PEM	<code>APP__JWT__PUBLIC_KEY_PEM="...jwt_public.pem..."</code>	Out-of-mesh / air-gapped. Overrides A. Must match the private key in 4.2.

The built-in fetcher speaks plaintext **HTTP** (the in-cluster east-west path). For HTTPS/off-cluster JWKS, use Option B.

4.4 Issuer & audience (must agree on both sides)

```
APP__JWT__ISSUER=https://identity.your-domain.com # NOT the identity.local default in prod
APP__JWT__AUDIENCE=ledger-platform
```

6 · Worker admin token

Guards the destructive DLQ-replay endpoint `POST /admin/dlq/replay/:topic`. Generate a random secret; the worker refuses to boot outside `local` if it is unset or left as `changeme`.

```
WORKER_ADMIN_TOKEN=$(openssl rand -hex 32)
WORKER_LEDGER_GRPC_ADDR=http://ledger:50052 # worker → ledger gRPC
```

7 · OAuth2 (optional — social login)

Only if you enable the OAuth login flow. Register an app with your provider (GitHub, Google, ...), set the callback to `https://identity.your-domain.com/oauth/callback`, and supply:

```
APP__OAUTH__CLIENT_ID=<from provider>
APP__OAUTH__CLIENT_SECRET=<from provider>
```

The shipped provider adapter is a demo stub — implement a real token-exchange + userinfo call before enabling OAuth in production (see `identity/adapters/outbound/oauth.rs`).

8 · Public hostname, TLS & ingress

- Point a DNS record at your ingress / load balancer (e.g. `api.your-domain.com`).
- Terminate TLS at the ingress (cert-manager / ACM / managed cert). The gateway itself serves plain HTTP behind it; HSTS is emitted for you.
- Set the `host` and `tls.secretName` in `deploy/k8s/base/ingress.yaml` to your domain and cert Secret.
- Make `APP__JWT__ISSUER` match your real identity hostname.

9 · Container registry (CI/CD)

Optional — only if you build & push images. The GitHub Actions workflow (`.github/workflows/docker.yml`) publishes to a registry. Provide, as repo/org secrets:

- Registry login (e.g. `GHCR_TOKEN`, or cloud `AWS_*` / `GCP_*` creds).
- Update the image refs in `deploy/k8s/base/*.yaml` from `ledger/<svc>:latest` to `<your-registry>/<svc>:<tag>`.

10 · Where the values go

Local dev

```
cp .env.example .env
# fill in the values, then:
just up          # or: docker compose \
  -f deploy/docker-compose.yml up --build
```

Production (Kubernetes)

Populate the `ledger-secrets` Secret (template: `deploy/k8s/base/secret.example.yaml`) via SOPS / sealed-secrets / an external secrets operator — never `kubectl apply` raw secrets. Keys map onto the env vars above; the migrate job applies schema.

```
# Apply schema (runs migrations for all three DBs):
kubectl apply -f deploy/k8s/base/migrate-job.yaml
# Deploy the stack:
kubectl apply -k deploy/k8s/overlays/production
```

11 · Production readiness checklist

- ☐ Managed Postgres reachable; `app` role + 3 databases created with a strong password
- ☐ Each service's `APP__DATABASE__URL` points at its own database (identity/ledger/worker)
- ☐ Redis reachable (with password/TLS as needed)
- ☐ Kafka/Redpanda brokers reachable; DLQ topics exist
- ☐ RSA keypair generated; `APP__JWT__PRIVATE_KEY_PEM` set on identity
- ☐ Gateway public-key delivery chosen (JWKS URL or static PEM) and verified end-to-end
- ☐ `APP__JWT__ISSUER` / `AUDIENCE` set to real values on both sides
- ☐ `WORKER_ADMIN_TOKEN` set to a strong random secret
- ☐ `RUN_ENV=production` set (enables fail-fast on insecure defaults)
- ☐ All secrets delivered via a secret manager, not committed or applied as raw manifests
- ☐ DNS + TLS ingress configured; hostname matches the JWT issuer
- ☐ `migrate-job` applied before the services start serving traffic
- ☐ OTEL collector endpoint set (or tracing intentionally disabled)
- ☐ Smoke test passes: `BASE_URL=https://api.your-domain.com k6 run tests/load/smoke.js`

Atlas ledger platform · Manual Setup & Secrets Guide · Apache-2.0 · This document lists only the values a human must supply; all wiring, resilience, and observability ship in the repo.